

TP — Patrones de Diseño

- Materia: Ingeniería de Software
- Tema: Patrones de Diseño Creacionales, Estructurales y de Comportamiento
- Patrones: Singleton · Strategy · Observer
- Modalidad: 1 trabajo por equipo
- Tiempo: 2 semanas desde la presentación del práctico
- Lenguaje: Cualquier lenguaje orientado a objetos (Java recomendado)

Descripción general

El objetivo de este trabajo es implementar tres patrones de diseño clásicos integrados en un sistema de transporte simulado. No alcanza con que el programa funcione: **la estructura del código debe reflejar claramente cada patrón**. Los tres patrones deben colaborar en un único programa ejecutable.

Aviso: No se recomienda el uso de IA generativa para escribir el código. Se valorará el estilo propio.

Parte 1 — Logger Global [Singleton]

Implementar un logger de aplicación usando el patrón **Singleton**. La clase debe garantizar que exista **una única instancia** durante toda la ejecución. Debe exponer cuatro métodos de logging, cada uno con un nivel y color de salida en la terminal usando códigos ANSI:

Método	Color	Prefijo
logWarning(String msg)	Amarillo	[WARN]
logDebug(String msg)	Verde	[DEBUG]
logInfo(String msg)	Gris	[INFO]
logError(String msg)	Rojo	[ERROR]

Requisitos:

- Constructor privado, instancia estática, método estático de acceso (`getInstance()`).
- La instancia se crea la primera vez que se llama a `getInstance()` (lazy initialization).
- Todos los métodos de log deben incluir un timestamp en el mensaje.

Parte 2 — Medio de Transporte [Strategy]

Modelar distintos medios de transporte usando el patrón **Strategy**. Cada medio de transporte es una estrategia intercambiable que implementa la misma interfaz. Se debe suponer un mismo origen-destino inventado para los distintos transportes. Los valores deben ser **dinámicos**: recalcularse con números aleatorios en cada llamada para simular variaciones por demanda y tránsito, a excepción del nombre del transporte.

Interfaz **TransportStrategy** :

```
getName()    → String    // nombre del transporte
getCost()    → double    // costo estimado del viaje
getDistance() → double    // distancia al origen en km
getETA()     → int       // tiempo estimado de arribo en minutos
```

Requisitos:

- Implementar al menos **3 estrategias concretas** (ej: Taxi, Colectivo, Bicicleta).
- Cada estrategia tiene su propia lógica de cálculo y rangos de variación distintos (un taxi es más caro que un colectivo, una bici tiene menos variación de precio, etc.). Los valores pueden ser simulados con un randomizer, poner umbrales de valor máximo y mínimo relativamente coherentes para apreciar las diferencias.
- El contexto que usa la estrategia debe poder cambiarla en tiempo de ejecución a través de una interfaz de consola.

Parte 3 — Monitor en Tiempo Real [Observer]

Implementar el patrón **Observer** para notificar cambios en los datos de un transporte. Un **TransportMonitor** actúa como *subject*: cada cierto intervalo recalcula los valores del transporte activo y notifica a todos sus suscriptores.

Interfaz **TransportObserver** :

```
onUpdate(TransportSnapshot snapshot)
```

Clase **TransportMonitor** (subject):

- Mantiene la lista de observers.
- Expone `subscribe(observer)`, `unsubscribe(observer)`.
- Tiene un método `start(intervalMs)` que lanza un loop: recalcular → construir snapshot → notificar a todos.

TransportSnapshot es un objeto de datos simple (o record en Java 16+) que encapsula el estado en un momento dado:

```
name      : String
cost      : double
distance  : double
eta       : int
```

Observers concretos

Se deben implementar **dos observers** con responsabilidades claramente separadas:

ConsolePrinter

- Imprime el estado completo del transporte en **cada** notificación.
- Usa `logInfo` para el encabezado y `logDebug` para los valores de distancia y costo.
- No contiene lógica de negocio ni umbrales: su única responsabilidad es mostrar los datos.

AlertObserver

- **No** imprime en cada ciclo: solo reacciona cuando los valores superan umbrales definidos.
- Usa `logWarning` si el costo supera un valor límite configurado.
- Usa `logError` si el ETA supera un tiempo límite configurado.
- Los umbrales se definen al construir el objeto (por constructor o setters).
- No sabe nada de `ConsolePrinter` y viceversa.

La separación entre estos dos observers ilustra el beneficio central del patrón: el mismo evento puede disparar reacciones completamente distintas en clases que no se conocen entre sí.

Integración — **main**

El `main` debe demostrar los tres patrones funcionando juntos:

1. Obtener la instancia del logger y loggear el inicio de la aplicación.
 2. Crear al menos dos estrategias de transporte distintas.
 3. Crear un `TransportMonitor` con una estrategia inicial.
 4. Suscribirse un `ConsolePrinter` y un `AlertObserver`.
 5. Llamar a `start()` y observar las actualizaciones en tiempo real por al menos 10 ciclos.
 6. Permitir al usuario cambiar la estrategia activa desde la consola sin detener el monitor.
-

Entregables

- ☐ Código fuente completo y compilable.
 - ☐ Al menos 3 estrategias de transporte.
 - ☐ Los dos observers (`ConsolePrinter` y `AlertObserver`) usando el logger.
 - ☐ Diagrama UML de clases (puede ser a mano escaneado) mostrando los tres patrones.
 - ☐ Un README con la evidencia y los resultados.
-

Preguntas orientadoras

1. **Singleton** — ¿Por qué un logger global es buen candidato para Singleton? ¿Qué problemas puede traer en aplicaciones multihilo? ¿Cómo se resolvería?
 2. **Strategy** — ¿Qué deberías hacer ¿modificar ó crear? si quisieras agregar un nuevo medio de transporte? ¿Qué principio SOLID describe esa propiedad del diseño?
 3. **Observer** — ¿Qué pasa si un observer tarda mucho en procesar la notificación? ¿Cómo se desacoplaría la velocidad del subject de la del observer? (Opcional)
 4. **Integración** — `ConsolePrinter` y `AlertObserver` usan el mismo logger pero de forma distinta. ¿Por qué eso es posible sin que se conozcan entre sí?
-